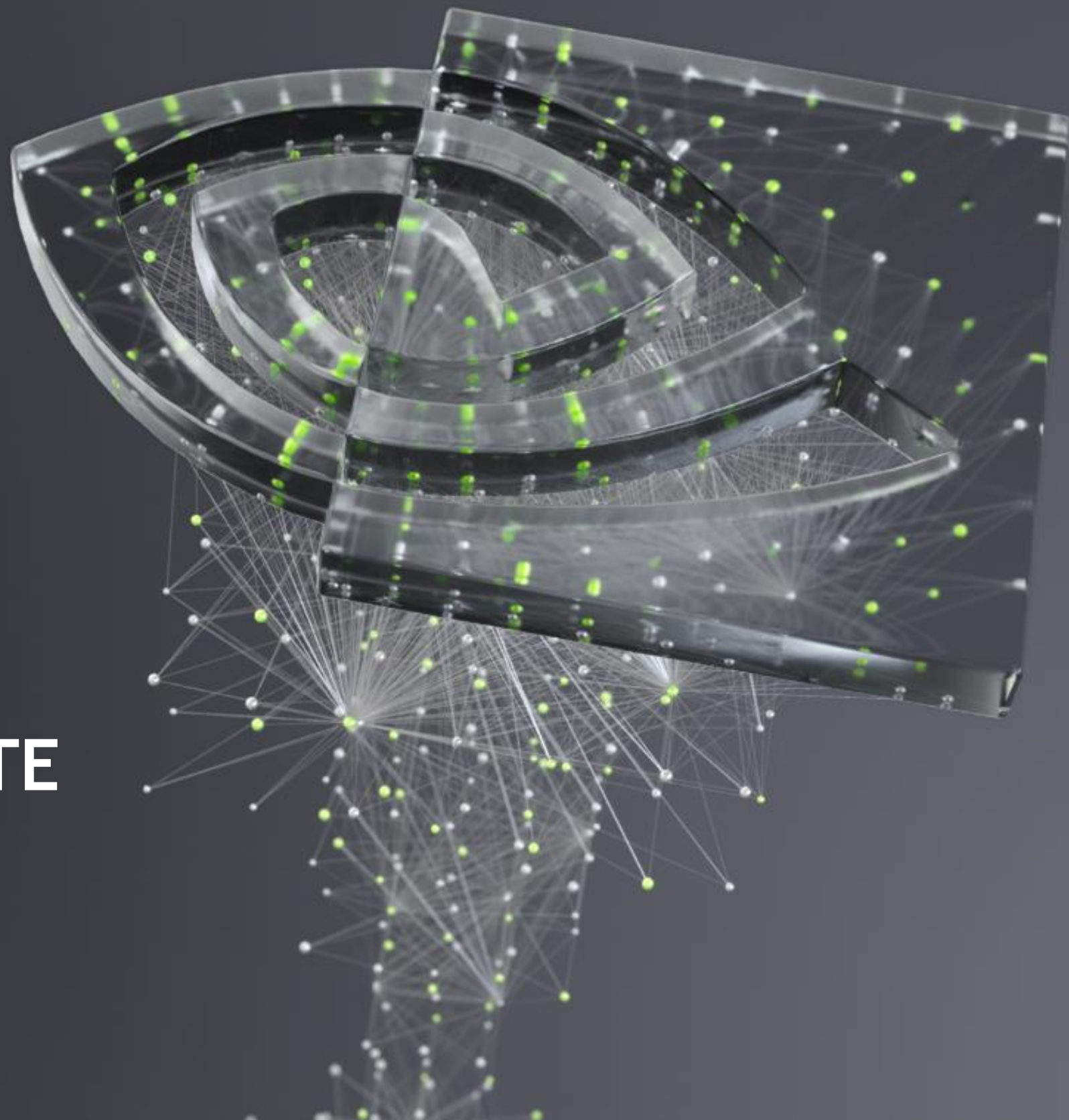




HOW CAN WE LIVE-MIGRATE ANY PCIE INTERFACE?

Max Gurtovoy | April 2021





AGENDA

Introduction

Virtualization, Virtual Machines, Live Migration

Why ?

Use cases and benefits

How ?

Migration software and HW pre-condition

What Next ?

Live Migration for Peer-to-Peer capable devices

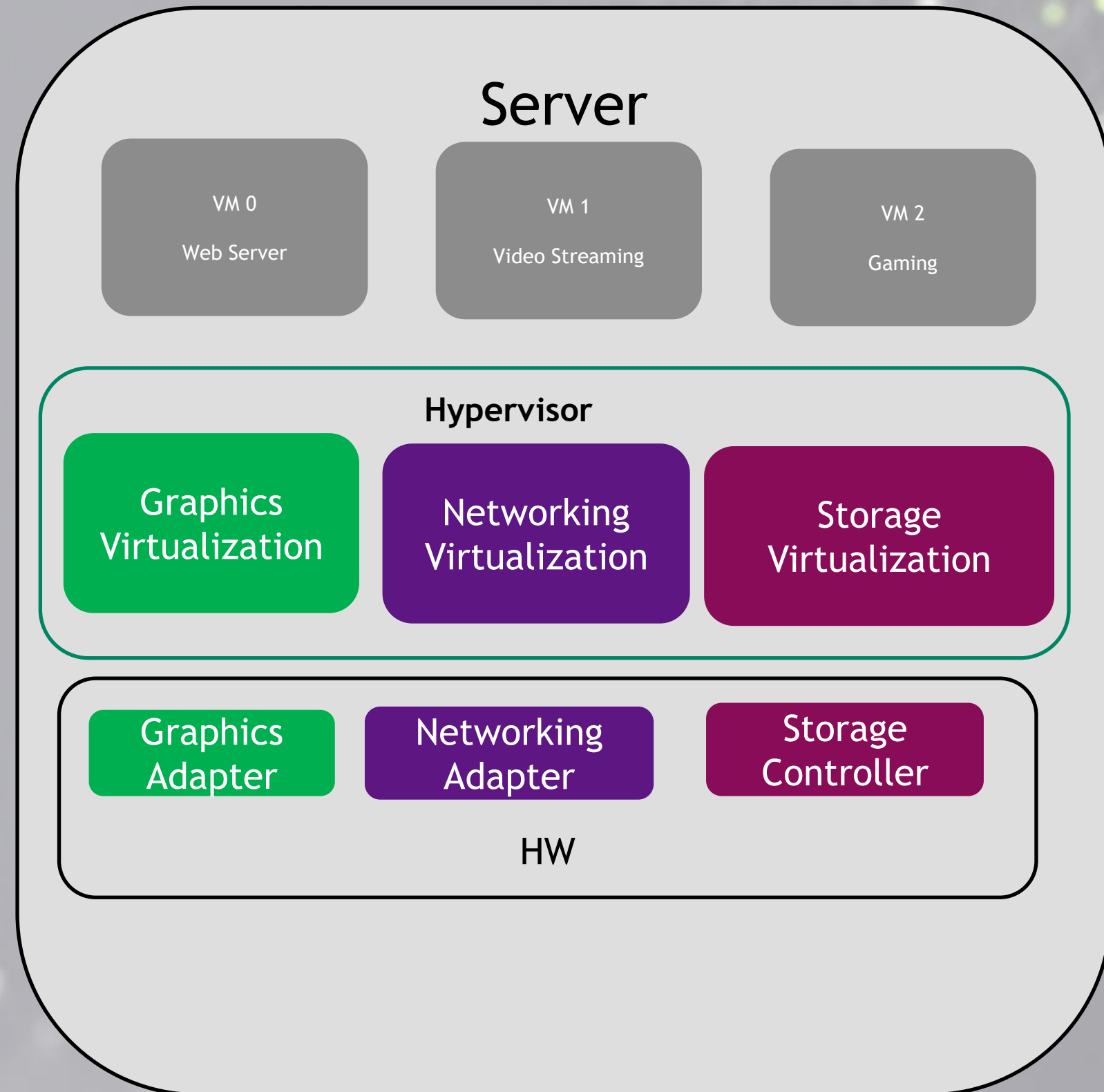
Demo

Virtio block PCI device Live Migration

INTRODUCTION

Virtualization

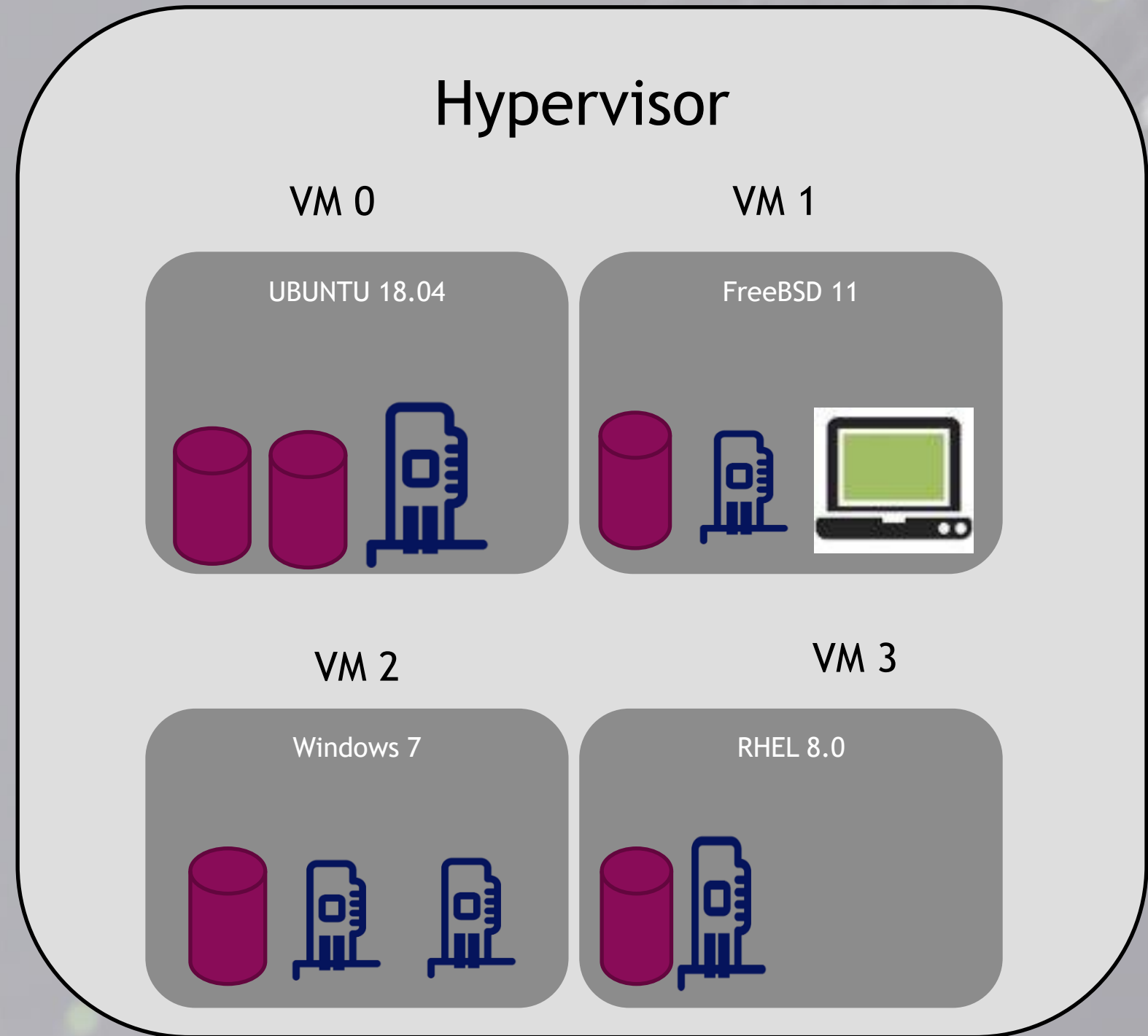
- SW (virtual) representation of a physical computer (VM, guest)
- Hypervisor to isolate/coordinate between VMs and physical HW
- Hypervisor allocates and manages resources (CPU, memory, storage) to VMs
- multiple virtual machines on a single physical machine
 - maximize the utilization of computer system resources
- Essential for cloud computing
- Examples:
 - Linux KVM + QEMU
 - Windows Hyper-V
 - VMWare ESXi



INTRODUCTION

Virtual Machines

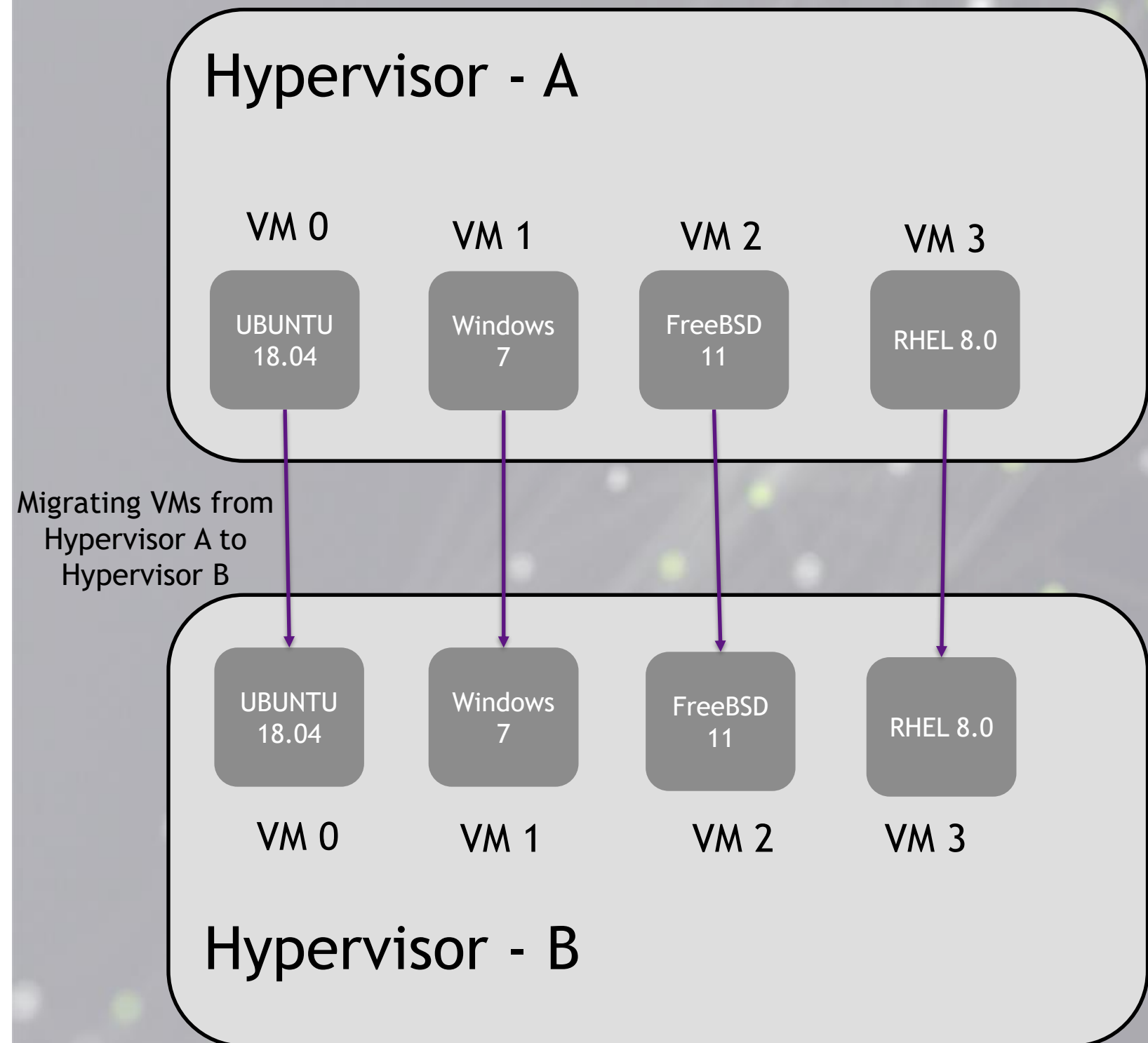
- Each virtual machine can run different OS and applications
- VMs are separated from each other
- QoS between VMs (amount of CPU, Memory, Storage, Networking BW)
- VMs are portable and migratable



INTRODUCTION

Live Migration

- One of the most important features of virtualization
- The movement of a virtual machine (VM) from one host to another.
- VM is powered up
- Applications running as usual





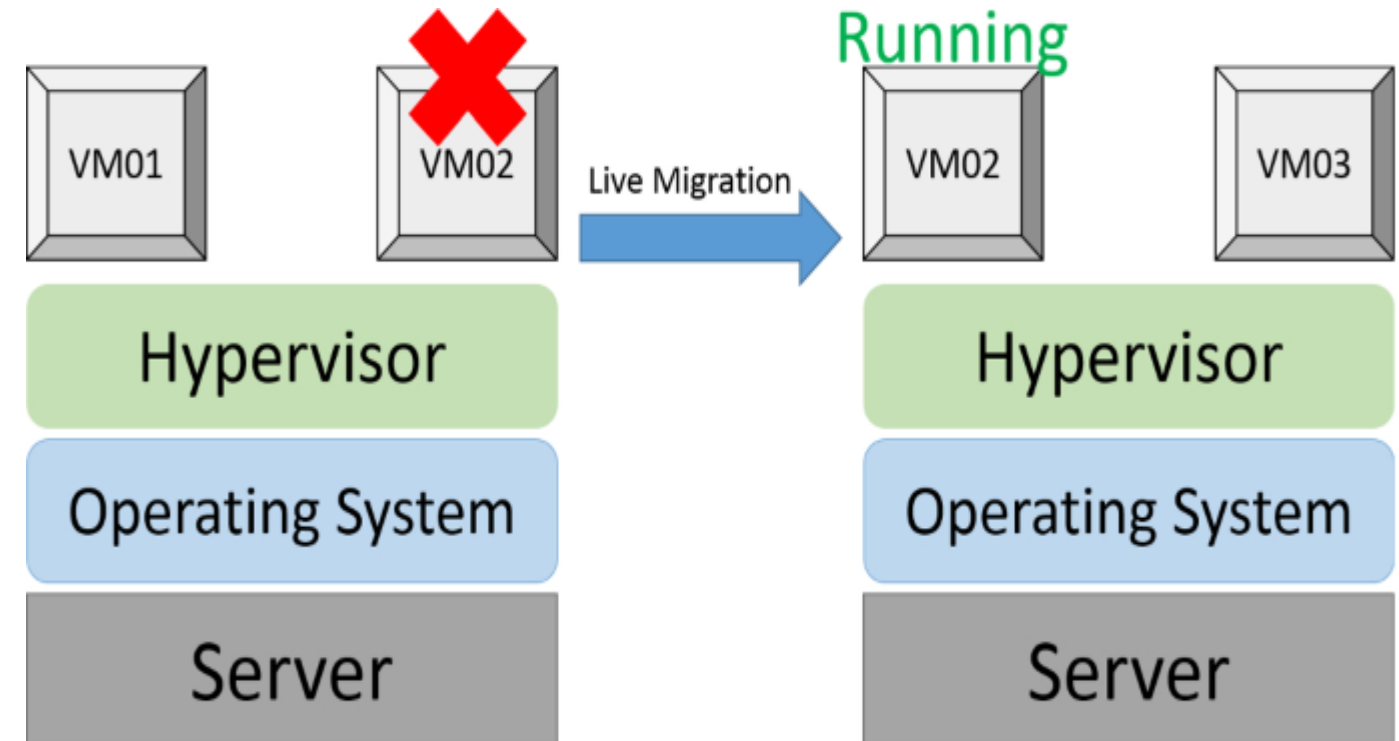
WHY ?

THE NEED FOR VIRTUALIZATION

- ▶ Cloud computing
 - ▶ Software defined everything
- ▶ RnD & DevOps
 - ▶ Testing/Automation/Verification
- ▶ Flexibility & Scalability
 - ▶ Multiple OS environments exist on same HW
 - ▶ Easy to deploy
- ▶ Cost savings
 - ▶ Reduce physical infrastructure footprint
 - ▶ Save maintenance costs
 - ▶ Save power
- ▶ Increase efficiency
 - ▶ Most users don't utilize all system resources
 - ▶ Share these resources with others
- ▶ Easy control
 - ▶ Updates done using UI/CMDLINE interface
 - ▶ No need to visit the lab
- ▶ Reduced downtime
 - ▶ HW errors can be easily recovered
 - ▶ Run/Start VM on new HW without effecting applications
 - ▶ Replicate/Deploy/Migrate VM

THE NEED FOR LIVE MIGRATION

- ▶ Load balancing
 - ▶ Split the job between multiple servers
- ▶ Power savings
 - ▶ Use fewer physical servers
- ▶ Maintenance
 - ▶ SW/FW/HW upgrades
- ▶ Availability of resources on different hypervisors
- ▶ Hypervisor failures



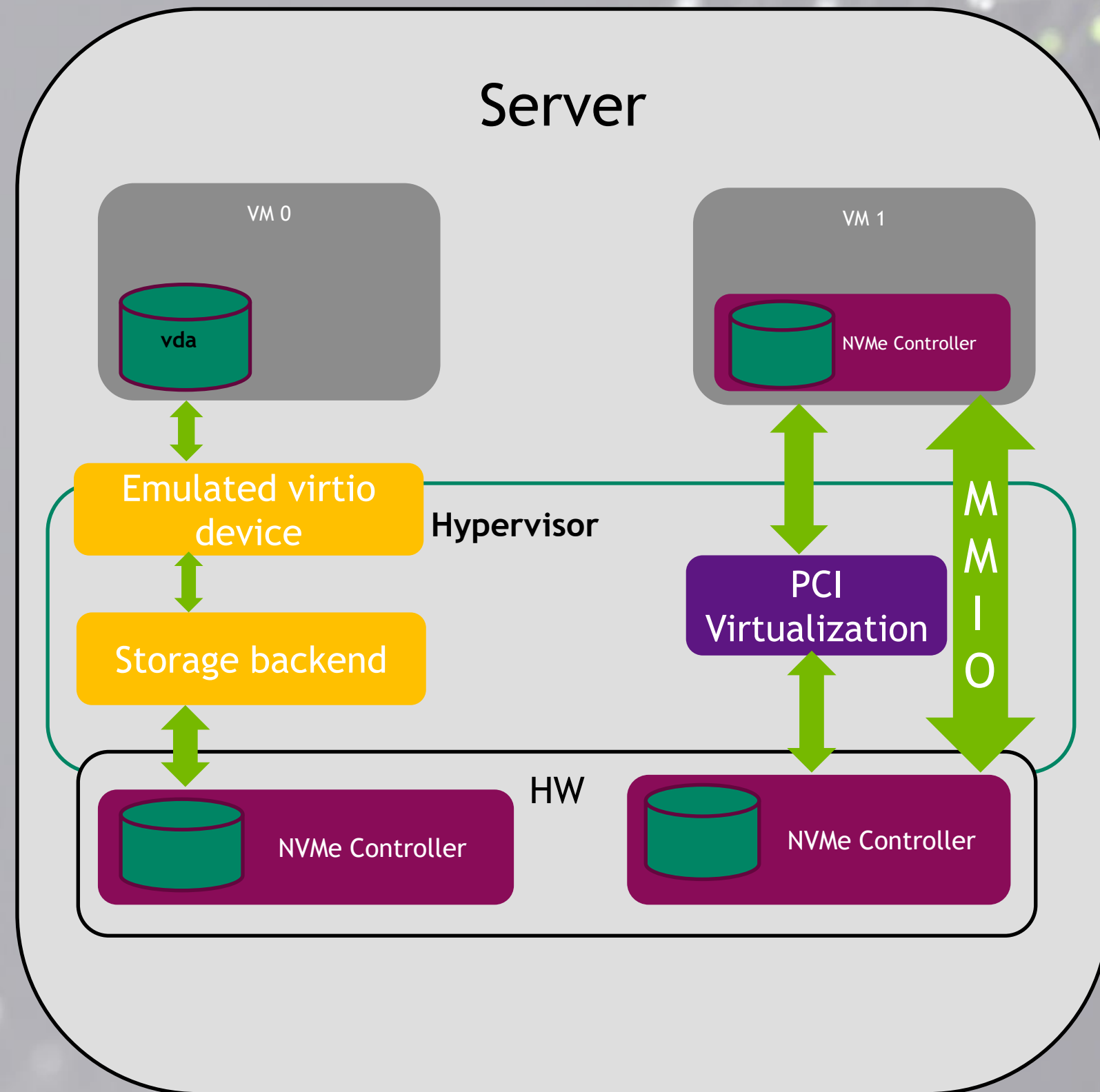
THE NEED FOR PCIE LIVE MIGRATION

Performance !!

Paravirt vs. SR-IOV

- Paravirt is a SW based device interface presented to the VM
 - Management is done by SW
 - IO processed by SW (trap-and-emulate)
 - Performance is non-optimized
- SR-IOV is a PCI-SIG specification
 - Portion of the physical device assigned directly to a VM
 - Aka Virtual Function (VF)
 - IO processed by HW
 - IOMMU protects memory and translate addresses
 - Performance is same as bare-metal system

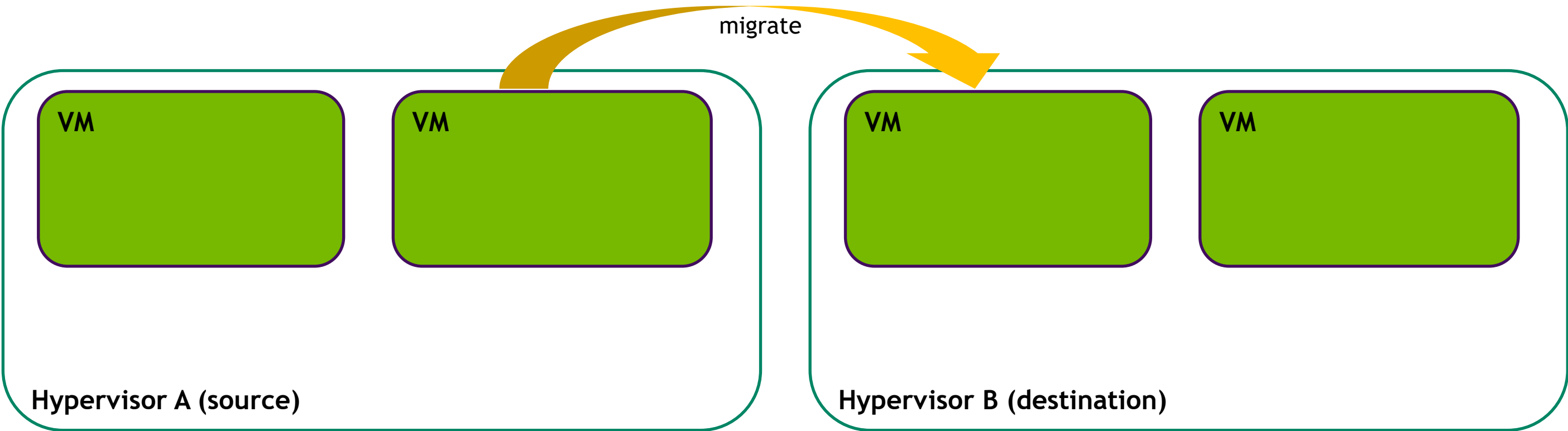
Running in a virtual environment on top of real HW





HOW ?

HOW DOES LIVE MIGRATION OF VMS WORK ?



Stages	Description
Reservation	Make sure necessary resources are available on Hypervisor B
Iterative Pre-Copy	First transfer the entire memory. Iteratively copy only new dirty pages
Stop-and-Copy	Suspend running VM on A. Remaining memory pages and final device state is sent to B. The copy to A is still primary. This is the stage that the VM is out of service (downtime should be minimized as possible)
Commitment & Activation	Hypervisor B successfully received all content and becomes primary host. Hypervisor A release VM state. All devices state resumed. Entire OS was migrated. Run VM on B.

LIVE MIGRATION CHALLENGES

- ▶ Minimize “total migration time”
 - ▶ This is the period between migration start until original VM is discarded
 - ▶ Use Pre-Copy migration
 - ▶ Use fast network (RDMA based ?)
- ▶ Minimize Downtime
 - ▶ This is the period in which service is unavailable
 - ▶ Visible to clients
 - ▶ Minimize to milliseconds level
- ▶ Migrate “non-friendly” devices
 - ▶ Paravirt devices are “friendly” and easy to migrate (emulated by SW and IO is trapped by the hypervisor)
 - ▶ SR-IOV devices are “non-friendly” (State is in HW, direct IO path between guest memory and HW)

LIVE MIGRATION PRE-CONDITIONS

- ▶ Migration SW is needed in order to manage migration process
- ▶ Each device must implement the following in order to be migratable:
 - ▶ Operational state change (quiesced/unquiesced/freeze/unfreeze)
 - ▶ State save/restore
 - ▶ Dirty pages tracking
- ▶ Each device must implement a communication channel towards the migration SW
 - ▶ This channel will be used to communicate the migration process
 - ▶ Control device states
 - ▶ Report on dirty pages

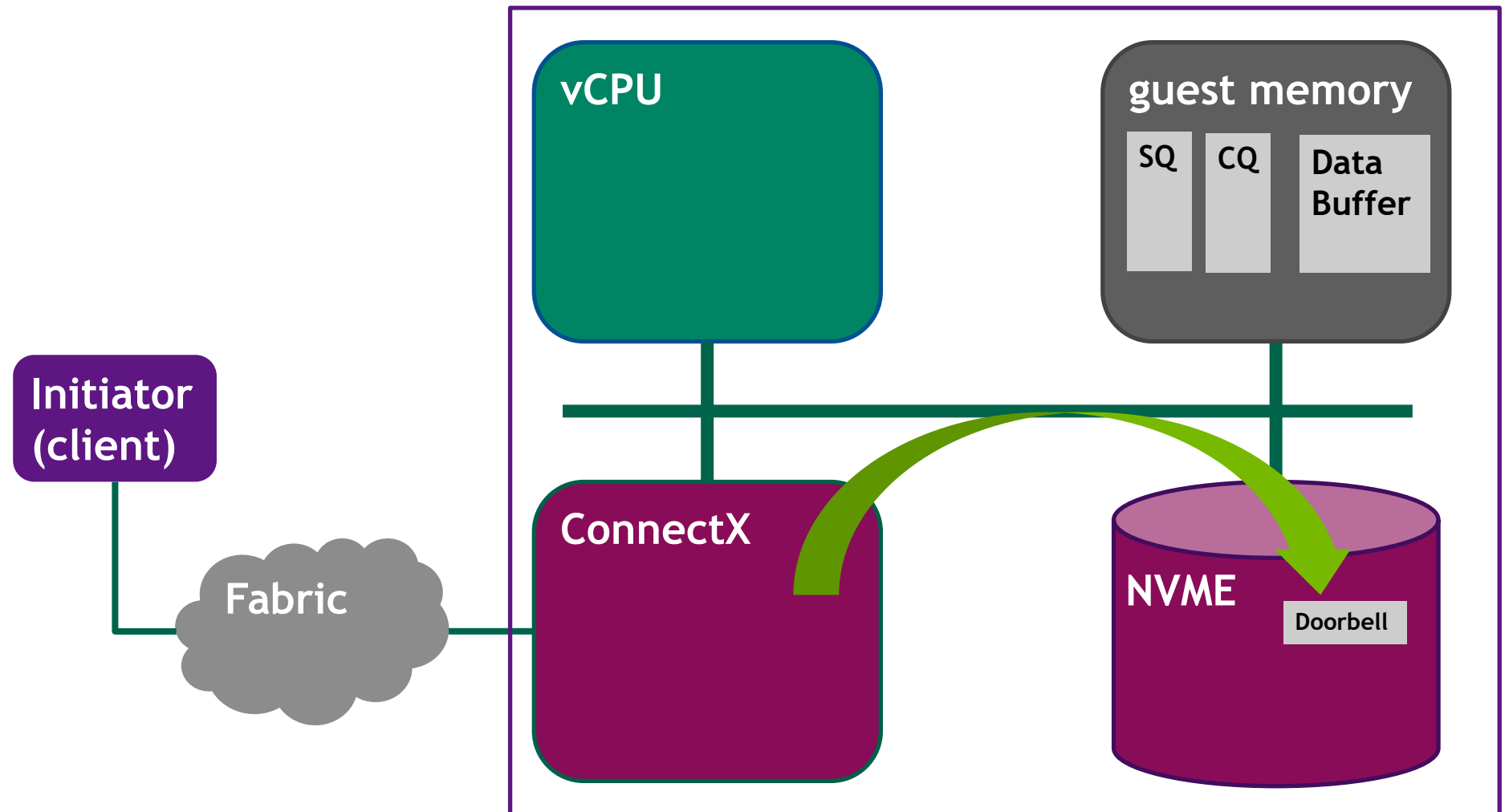


WHAT NEXT ?

PEER-2-PEER LIVE MIGRATION

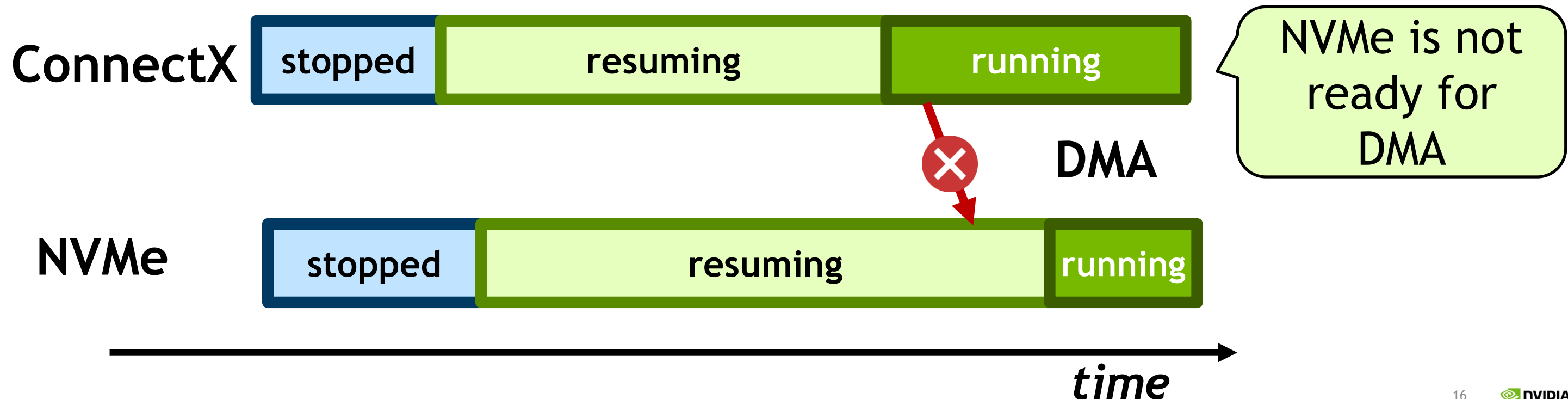
No CPU cycles during IO path !

- ▶ DMA transfers between 2 devices on the PCI bus
- ▶ For example:
 - ▶ NVMf target offload using NVIDIA ConnectX adapter and an NVMe device
 - ▶ GPU direct using NVIDIA GPUs and NVIDIA network adapters
 - ▶ GDS - GPU direct Storage using NVIDIA GPU and an NVMe device



PEER-2-PEER LIVE MIGRATION CHALLENGES

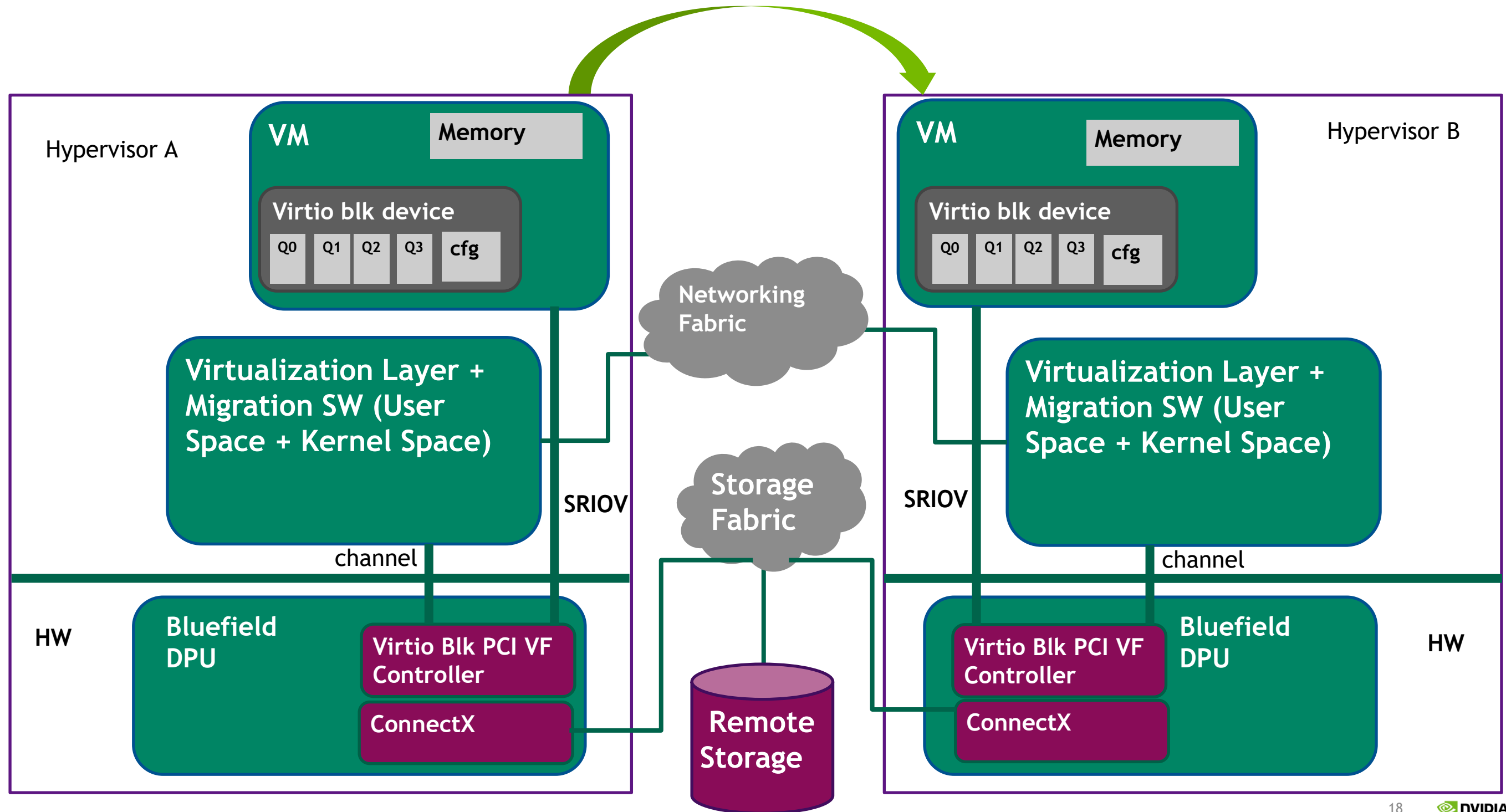
- Unquiesced device can issue DMA transaction:
 - To guest memory
 - To PCI device
- Target PCI device might be freezed (during resuming or during saving phase)
- Migration SW must manage state machine correctly for P2P devices (e.g unquiesce device only after all devices were un-freezed during resuming phase)





DEMO !

VIRTIO BLOCK DEVICE LIVE MIGRATION



GTC 2021 PCI Live Migration Demo

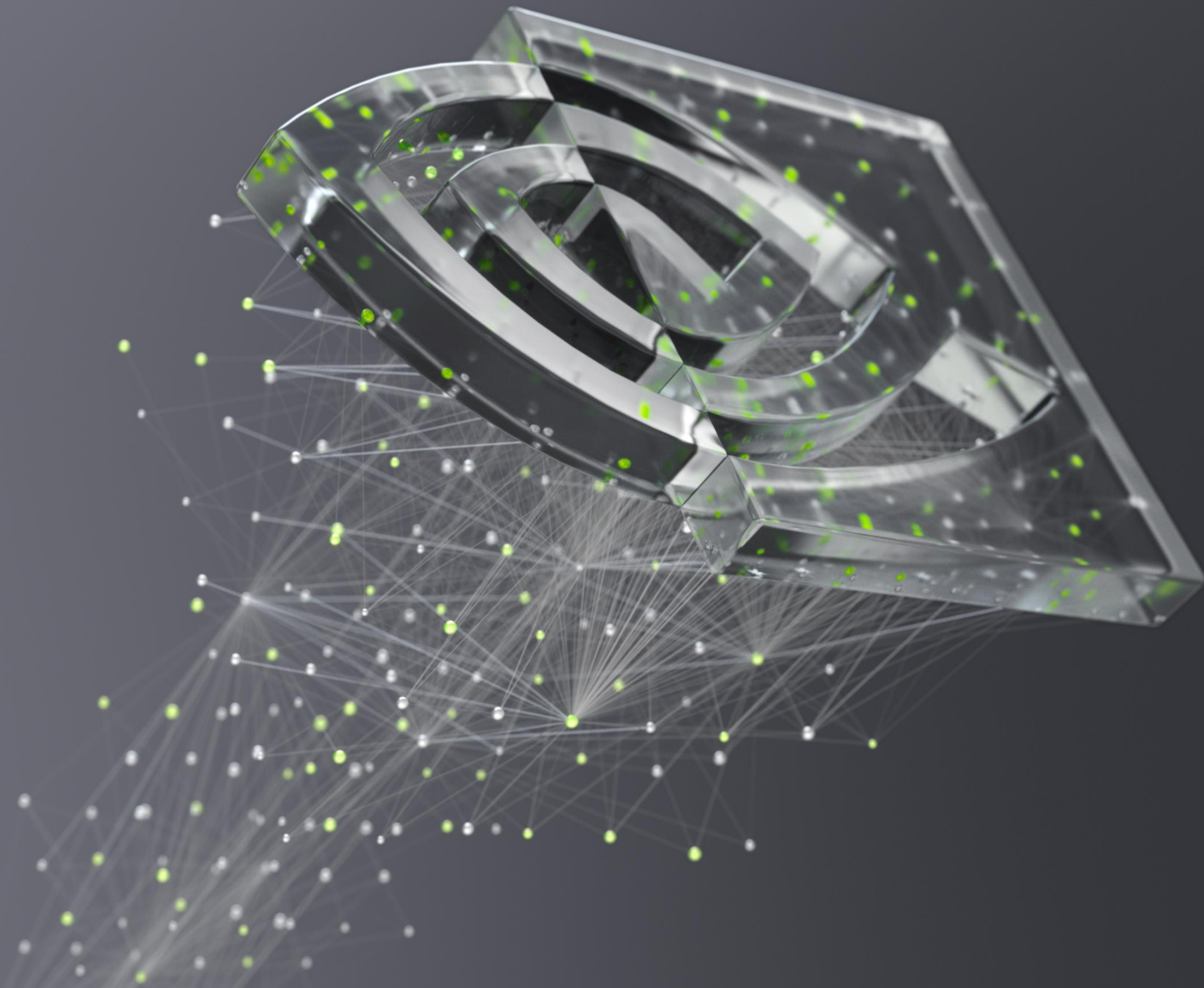
2021-03-26 10:01 UTC

Recorded by

Max Gurtovoy

Organized by

Max Gurtovoy



nvidia